

Smart Canteen - Next Steps Task Plan

Phase 2 continuation - two parallel workstreams assigned to two teammates. Completion criteria: endpoints fully working, all tests passing, code committed.

Overview of the Two Steps

The backend currently has working models (users, categories, menu_items, orders, order_items, inventory), the Alembic migration, and a fully functional JWT auth system. These two tasks add the core business APIs on top of that foundation. They can be developed in parallel since they touch different files.

Task	Owner	Scope
Step 1	Team Member 1	Menu & Category CRUD endpoints
Step 2	Team Member 2	Order & Inventory endpoints

Shared rules for both tasks are listed at the end of this document. Run everything from the backend/ folder using the venv Python.

Step 1 - Menu & Category CRUD (Team Member 1)

Build RESTful CRUD endpoints for categories and menu items. Admin users manage the menu; customers can read it. Follow the existing layered pattern: schemas -> repository -> service -> router.

1.1 Files to create / modify

- **app/schemas/menu.py** - CategoryCreate/Update/Out, MenuItemCreate/Update/Out (with pydantic validation: price ≥ 0 , stock ≥ 0 , min_length on names).
- **app/repositories/menu_repository.py** - CRUD helpers for Category and MenuItem.
- **app/services/menu_service.py** - business logic (409 if category name duplicate, 404 if not found).
- **app/api/v1/menu.py** - router with prefix /categories and /menu-items.
- **app/api/v1/__init__.py** - register your new router.

1.2 Endpoints to implement

- GET /categories - public, returns all categories with their menu items.
- POST /categories - admin only (depend on get_current_admin).
- GET /categories/{id} - public, 404 if missing.
- PATCH /categories/{id} - admin only.
- DELETE /categories/{id} - admin only, cascades to menu items.
- GET /menu-items - public, optional filters: ?category_id= & ?is_available=true.
- POST /menu-items - admin only, validates category exists.
- GET /menu-items/{id} - public, 404 if missing.
- PATCH /menu-items/{id} - admin only.
- DELETE /menu-items/{id} - admin only.

1.3 Hints

Reuse `app.api.deps.get_current_user / get_current_admin`. Keep `image_url` nullable. Use `response_model=CategoryOut` so passwords/relations never leak. Import the models from `app.models` - they are already mapped in the DB.

1.4 Verification

Create a category, then a menu item under it, update its price, list with the category filter, delete it, and confirm the same flow returns 404 afterwards. Register an admin user in the DB (`role='admin'`) or temporarily promote one to test the admin endpoints. Run the full test suite:

```
backend\venv\Scripts\python.exe -m pytest tests -v
```

Step 2 - Order & Inventory Endpoints (Team Member 2)

Build the ordering flow: a customer creates an order with items, the total is computed server-side, and inventory stock is deducted. Also expose inventory management endpoints for the admin.

2.1 Files to create / modify

- **app/schemas/order.py** - OrderItemIn (menu_item_id, quantity >= 1), OrderCreate (list of items), OrderItemOut, OrderOut (with computed total).
- **app/repositories/order_repository.py** - CRUD + queries for orders and order items.
- **app/services/order_service.py** - totals, stock validation, stock deduction, order status transitions.
- **app/api/v1/orders.py** - router with prefix /orders.
- **app/api/v1/inventory.py** - router with prefix /inventory.
- **app/api/v1/__init__.py** - register both routers.

2.2 Endpoints to implement

- POST /orders - customer only (current user), creates order + order items, computes total, checks each menu_item stock, deducts stock, returns the created order.
- GET /orders - customers see their own orders only; admins see all.
- GET /orders/{id} - owner or admin only, 404 if missing, 403 if not allowed.
- PATCH /orders/{id}/status - admin only; statuses: pending, confirmed, completed, cancelled.
- GET /inventory - admin only, lists stock per menu item.
- PATCH /inventory/{menu_item_id} - admin only, sets or adds stock (restock).

2.3 Hints

Reject the whole order with 409 if any item is out of stock (rollback, no partial deduction). Compute prices from menu_items.price at order time - never trust client prices. Commit once, at the end, after all stock updates succeed (single transaction). Use selectinload or eager loading for order items to avoid N+1 queries.

2.4 Verification

Create a menu item with stock=5, place an order for quantity 2, confirm stock becomes 3, total equals 2 x price. Order 10 more -> expect 409 and stock unchanged (still 3). Complete the flow to 'completed'. A non-owner user must get 403 on GET /orders/{id}. Run the full test suite:

```
backend\venv\Scripts\python.exe -m pytest tests -v
```

Shared Rules for Both Tasks

Code conventions

- Layered architecture: schemas -> repositories -> services -> routers. Keep business logic out of routers.
- No comments in code unless truly necessary; follow the style of existing files.
- Routers use APIRouter with explicit tags and response_model.
- Never expose hashed_password or internal fields in responses.

Error handling

- 404 for missing resources, 403 for unauthorized roles, 409 for conflicts, 422 handled automatically.
- Raise HTTPException in services with clear detail messages - the global handler formats them.
- Validate with pydantic schemas (min_length, ge=0 for money/quantities).

Testing & delivery

- Add tests under tests/ (see tests/test_auth.py as template, including cleanup of test rows).
- All tests must pass before merging: `python -m pytest tests -v`.
- After finishing, commit and push your branch, then report the endpoints you implemented.

Deadline & acceptance: both steps must be fully working end-to-end, verified with the live server on port 8000, with no errors and no failed tests.

Generated for the Smart Canteen team - Phase 2 workstreams.